A Transformer-Based Contrastive Semi-Supervised Learning Framework for Automatic Modulation Recognition

Weisi Kong, Xun Jiao, Yuhua Xu, Bolin Zhang, and Qinghai Yang, *Member, IEEE*

***Abstract*—The application of deep learning improves the processing speed and the accuracy of automatic modulation recognition (AMR). As a result, it realizes intelligent spectrum management and electronic reconnaissance. However, deep learning-aided AMR usually requires a large number of labeled samples to obtain a reliable neural network model. In practical applications, due to economic costs and privacy constraints, there is a small number of labeled samples but a large number of unlabeled samples. This paper proposes a Transformer-based contrastive semi-supervised learning framework for AMR. First, self-supervised contrastive pre-training of the Transformer-based encoder is completed using unlabeled samples, and data augmentation is realized through time warping. Then, the pre-trained encoder and a randomly initialized classifier are fine-tuned using labeled samples, and hierarchical learning rates are employed to ensure classification accuracy. Considering the problems of applying Transformer to AMR, a convolutional transformer deep neural network is proposed, which involves convolutional embedding, attention bias, and attention pooling. In experiments, the feasibility of the framework is analyzed through linear evaluation of the framework components on the RML2016.10a dataset. Also, the proposed framework is compared with existing semi-supervised methods on RML2016.10a and RML2016.10b datasets to verify its superiority and stability.**

***Index Terms*—Automatic modulation recognition, semi-supervised learning, contrastive learning, transformer network.**

I. Introduction

VARIOUS wireless terminal devices greatly enrich daily life, but the management and utilization of limited wireless communication resources are facing a significant challenge. To improve the spectrum utilization and throughput, cognitive radio technology has been proposed, which adaptively adjusts communication parameters by sensing the surrounding electromagnetic environment. As an important component of the communication parameters, the modulation type of received signal is identified through automatic modulation recognition (AMR) [1]. In military applications such as reconnaissance and detection, AMR is adopted to judge the working parameters of the adversary's electronic equipment for formulating the next plan of action.

Traditional AMR methods include the maximum likelihood method based on hypothesis testing and the feature extraction method based on expert knowledge. However, both methods are computationally complex and time-consuming, and require manual settings and interventions [2]. With the rapid development of artificial intelligence, AMR methods based on deep learning (DL) have been extensively studied, which improves recognition efficiency and accuracy [3]. Typically, the original discrete signal is preprocessed into an in-phase quadrature (IQ) vector [4], [5], [6], [7], [8], an amplitude-phase (AP) vector [9], [10], a Fourier transform (FT) vector [11], or a time-frequency image (TFI) [12], [13]. Then, the encoder based on convolutional neural network (CNN), recurrent neural network (RNN), or Transformer is applied to extract signal features. Finally, the recognition result is output by the classifier.

Existing DL-based AMR methods have improved recognition accuracy by optimizing the model design, but network training requires a large number of samples to ensure model performance. Signal samples are easily captured in practical military and civilian applications, but due to economic, privacy, and intelligence constraints, there are only a few samples with high-quality annotated labels. Semi-supervised learning can be used for AMR in the case of a large number of unlabeled samples and a small number of labeled samples [14]. The key to semi-supervised learning is to improve the availability of unlabeled samples. The existing semi-supervised AMR frameworks mainly adopt pseudo-labels generated for unlabeled samples to train the network model [15], [16], [17]. However, this method cannot provide reliable pseudo-labels when the amount of labeled samples is small.

In this paper, an efficient contrastive semi-supervised AMR framework is proposed. First, data augmentation is performed on the unlabeled IQ signal, and the encoder is pre-trained by using contrastive learning (CL). Then, the pre-trained encoder and a randomly initialized classifier are combined into a network model for modulation recognition, and the model is fine-tuned using a small amount of labeled samples. CL is an individual discrimination approach of self-supervised learning in the computer vision (CV) field, and it has low complexity

Manuscript received 28 July 2022; revised 16 November 2022 and 21 February 2023; accepted 28 March 2023. Date of publication 5 April 2023; date of current version 14 August 2023. The associate editor coordinating the review of this article and approving it for publication was Z. Qin. *(Corresponding author: Weisi Kong.)*

Weisi Kong and Qinghai Yang are with the State Key Laboratory on ISN, School of Telecommunications Engineering, Xidian University, Xi'an 710071, Shaanxi, China (e-mail: kongsi0627@163.com; qhyang@xidian.edu.cn).

Xun Jiao, Yuhua Xu, and Bolin Zhang are with the Academy of Military Sciences, Beijing 100000, China (e-mail: blzhang@std.uestc.edu.cn).

Digital Object Identifier 10.1109/TCCN.2023.3264908

2332-7731 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Rowan University Libraries. Downloaded on November 03,2025 at 22:28:36 UTC from IEEE Xplore. Restrictions apply.

and fast convergence speed compared with pixel-level generative methods [18]. The goal of CL is to reduce the feature distance between different augmented views of an image while increasing the distance between augmented views from different images [19]. CL can be used in semi-supervised AMR to improve the utilization of unlabeled samples. According to the simulation results, the SimCLR [20] model is applied in this study. Self-supervised pre-training enables the encoder to have the ability of general feature extraction. Therefore, the pre-trained encoder is fine-tuned by using a smaller learning rate than the classifier.

It is worth mentioning that the collected signals are affected by random noise, multi-path fading, frequency offset, and other factors. To improve the performance of recognition, it is necessary to adopt appropriate data augmentation and encoder to extract effective feature information from low-dimensional IQ signals.

Data augmentation is a transformation that improves the generalization and stability of the network without changing the fundamental characteristics of samples. Literature [21] compared rotation, reverse and adding noise with AP vector as an input and the long short-term memory (LSTM) model as a feature extractor. Three basic data augmentation methods were analyzed from the perspective of constellation diagram of modulated signals, and the rotation achieved best results. In the semi-supervised AMR, the augmentation methods of generative adversarial [15], adding noise [17] and rotation [22] were applied. Different from above methods, considering the temporal correlation of IQ signals, various data augmentation methods are compared from the perspective of time series in this study. Comparison results show that the time-warping achieves better results.

The Transformer model for natural language processing was first proposed in [23]. Subsequently, the vision Transformer (ViT) model for the CV field was proposed in [24]. Nowadays, Transformers with global modeling and dynamic parameters have been used in many artificial intelligence applications. In literature [6], [7], [8], the Transformer-based AMR models were proposed. However, they cannot integrate effectively the characteristics of modulated signals and Transformer to improve the accuracy of recognition. Considering the characteristics of IQ signals, there are three problems with the existing Transformer-based AMR models:

1) In [6], [7], [8], the 2D IQ signal is projected into a high-dimensional space by embedding and then fed into a Transformer model for feature extraction. The Transformer encoder is good at global dependency modeling, but it is weak at extracting local information, so it requires more samples to achieve better performance. The methods of linear embedding and single-layer convolutional embedding are difficult to obtain the local dependence of IQ signals. 2) The self-attention with parallel computing requires positional encoding to provide position information of sequence. Different from images with a fixed range of pixel values and texts with a fixed range of coding values, the value of IQ signal is uncertain. The way to superimpose the positional encoding after the embedded sequence results in the masking of information in signals with a small magnitude and the loss of position information in high-level Transformer blocks. 3) The Transformer block does not change the length of input sequence, and the output is an encoded sequence with global dependencies modeled. In the recognition task, the encoded sequence needs to be transformed into a feature vector to represent the IQ signal, and the class probability is output by the classifier. However, the methods of flattening [6], pooling [7], and learnable class vector [8] will cause information loss.

This paper proposes an efficient Transformer-based encoder for the above-described problems. Specifically, multiple convolutional groups with the ability of local bias induction are used for embedding; meanwhile, attention bias that provides relative positional information is added to each self-attention mechanism; besides, the attention pool that adaptively weights the encoded sequence is placed at the output of the last Transformer block. The classifier composed of full connection layer and the Transformer-based encoder constitute a convolutional Transformer deep neural network (CTDNN).

The contributions of this paper are summarized as follows:

- A Transformer-based contrastive semi-supervised learning framework for AMR (TcssAMR) is proposed. In the pre-training stage involving unlabeled samples, an encoder with general feature extraction capability is obtained by applying CL; in the fine-tuning stage involving labeled samples, the network consisting of a pre-trained encoder and an initialized classifier is updated with hierarchical learning rates.
- The effect of various data augmentation methods on IQ signals is compared from the perspective of time series, and time-warping is applied to TcssAMR according to the experimental results. Additionally, considering the problems in applying Transformers to AMR, an effective Transformer-based encoder is proposed, which involves convolutional embedding, attention bias, and attention pooling. The encoder and classifier constitute the proposed CTDNN.
- In experiments, detailed linear evaluation of the components of TcssAMR is performed on the RML2016.10a dataset in terms of the data augmentation, encoder, and CL model. Moreover, TcssAMR is compared with other AMR methods on the RML2016.10a and RML2016.10b datasets. The experimental results indicate that TcssAMR effectively improves recognition accuracy under semi-supervised conditions.

The rest of this paper is organized as follows. Section II discusses the related works in terms of supervised AMR, semi-supervised AMR, and self-supervised learning. Section III briefly introduces the wireless signal model and the semi-supervised AMR problem. Section IV describes the structure and principle of the proposed framework in detail. Section V presents the analysis of experimental results. Finally, Section VI concludes the paper.

II. Related Works

A. Supervised Learning-Based AMR

In [4], O'Shea et al. established the RML2016.10a dataset equivalent to a real wireless communication system and

Authorized licensed use limited to: Rowan University Libraries. Downloaded on November 03,2025 at 22:28:36 UTC from IEEE Xplore. Restrictions apply.

constructed a CNN model composed of convolutional layers and fully connected layers to classify IQ signals. Considering the temporal correlation of signals, literature [5] proposed a convolutional long short term deep neural network (CLDNN) model combining CNN and LSTM. In [9], the received signal was converted to the AP representation, and the LSTM model was applied to AMR. In [10], the gated recurrent unit (GRU) with fewer parameters and lower complexity than LSTM was further applied, and a gated recurrent residual neural network was proposed in combination with ResNet. Although RNN networks such as LSTM and GRU can extract features from time series, the serial structure of RNNs suffers from the problems of memory forgetting and low efficiency. In contrast, the Transformer involves self-attention mechanism can model correlation of global features through parallel computation [23]. The Transformer-based MCFormer model was proposed in [7]. In this model, the IQ signal is taken as a 1-D image with a depth of 2 and fed into a convolutional layer with a kernel size of $k \times 1$ for embedding. Then, the encoding vector is obtained after pooling and flattening the output of the Transformer encoding block. In [8], the IQ sequence was divided into multiple short sequences of equal length, and the modulation recognition was realized by ViT in the form of an image. The accuracy of IQ, AP, and FT vectors in CNN [4] was analyzed in [11], and it was found that the AP vector achieved higher accuracy at high signal-noise ratio (SNR), while the IQ vector achieved higher average accuracy at all SNRs. The same results were obtained on the convolutional gated recurrent unit deep neural network (CGDNN) and CLDNN models in [25].

The received signal can also be represented as an image through time-frequency transformation. In [12], the pre-trained ResNeXt network was used as the backbone, and adaptive channel and spatial attention modules were embedded to fully extract the intrinsic features of signal. In [13], a time-frequency attention mechanism was proposed to extract features from channels, frequency, and time. Although AMR using TFI can capture the time-frequency features of signal and directly benefit from CV models, high-resolution image representations also result in larger computational complexity. Therefore, the IQ vector is used as the input of the network in this paper.

B. Semi-Supervised Learning Based AMR

In [15], a GAN-based recognition method was proposed, in which the generator was used to generate fake samples while the discriminator was used to judge the authenticity and modulation type of samples. The unlabeled samples were passed through the discriminator to generate pseudo-labels for supervised training. In [16], a small amount of labeled samples was used for pre-training, and the pseudo-labels of unlabeled samples were determined based on confidence. In this way, the dataset was extended to retrain the designed CNN model. In [17], a semi-supervised signal recognition convolutional neural network (SSRCNN) with multiple loss functions was proposed, where the Kullback-Leibler (KL) divergence loss and cross-entropy loss were introduced for unlabeled samples. The above methods cannot provide reliable pseudo-labels for unlabeled samples when the amount of labeled samples is small. In contrast, self-supervised CL was used in [22] to pretrain the encoder with unlabeled samples, and positive pairs were constructed through rotation augmentation. After pretraining, the encoder parameters were frozen, and the classifier was trained with labeled samples. This study is the first to apply CL to semi-supervised AMR, but it does not fully utilize unlabeled and labeled signals to improve the performance of model, and in-depth analysis and comparison are lacking.

C. Self-Supervised Learning

Self-supervised learning which includes generative and discriminative methods, learns feature representations from the data without labels. Generative methods mainly employ the architectures of GAN [26] and AutoEncoder [27] to model the pixel-level relationship between generated samples and original samples. This type of method suffers from high complexity and slow convergence. In contrast, discriminative CL methods model the correlations between different instances through simple individual discriminative structures. In [28], an individual discriminative pretext task that treated each sample as a class was proposed, and self-supervised representation learning was implemented with a noise contrastive estimation (NCE) loss. Momentum contrast (MoCo) [29] maintained a queue of negative samples as a memory bank and used a momentum encoder to update network parameters to obtain consistent feature representations. In [20], an end-to-end feature learning framework called SimCLR was proposed, which used data in batches. The model added nonlinear fully connected layers between the encoder and contrastive loss as a projection head, thus improving the feature representation capability of the encoder. Compared to MoCoV1 [29], MoCoV2 [30] combined the projection head in SimCLR to reduce hardware costs and achieved better results. MoCoV3 [31] further used ViT as an encoder and explored the impact of embedding on network performance. Different from the above approaches, BYOL [32] achieved a more efficient feature representation in the form of predictions, without negative sample pairs.

III. Signal Model and Problem Formulation

This section briefly describes the wireless signal model and the problem formulation of semi-supervised AMR.

A. Signal Model

In communication systems, the bit stream of information is mapped to a discrete complex symbol s_k through encoding and modulation. After pulse shaping filter and digital-to-analog conversion, the discrete symbol is further transformed into an analog continuous signal $s_b(t)$, which can be expressed as:

$$s_b(t) = a(t)\exp(j2\pi f(t)t + \varphi(t) + \varphi_0), \tag{1}$$

where $a(t)$, $f(t)$, and $\varphi(t)$ are the instantaneous amplitude, instantaneous frequency, and instantaneous phase, respectively; φ_0 is the initial phase of the signal. After the spectrum shifting with sub-carrier frequency f_c, the transmission signal

Authorized licensed use limited to: Rowan University Libraries. Downloaded on November 03,2025 at 22:28:36 UTC from IEEE Xplore. Restrictions apply.

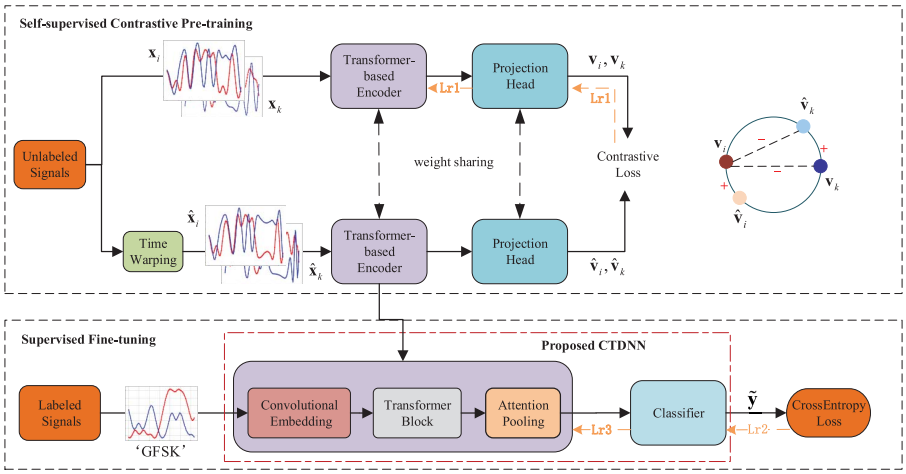


Fig. 1. The framework of TcssAMR.

$s(t)$ is obtained as:

$$s(t) = \Re\left\{ s_b(t) e^{j2\pi f_c t} \right\}. \tag{2}$$

The signal sent by the transmitter reaches the receiver via wireless channel. The received signal $r(t)$ is:

$$r(t) = s(t) * h(t,\tau) + n(t), \tag{3}$$

where $h(t,\tau)$ is the multi-path fading channel, and $n(t)$ is the additive white Gaussian noise with a mean of 0 and a variance of σ^2. The received signal $r(t)$ is processed via amplification, down-conversion, and low-pass filtering, and the discrete signal $r(l)$ is obtained through analog-to-digital conversion. Assuming that the sampling frequency is $f_s = 1/T_s$ and the sampling period is T_p, the discrete-time series $\mathbf{r}$ with a length of L can be obtained:

$$\mathbf{r} = [r[0], r[1], \ldots, r[L-1]]^T \in \mathbb{C}^L, \tag{4}$$

where $r[l]$ can be expressed as the combination of the in-phase component $r_I[l]$ and the quadrature component $r_Q[l]$, i.e.,

$$r[l] = r_I[l] + jr_Q[l], l = 0, 1, \ldots, L-1. \tag{5}$$

In DL-based AMR, the original discrete signals are usually preprocessed into an IQ vector, an AP vector, a TFI, or an FT vector. In this study, the IQ representation is used directly to avoid the amount of computation and achieve higher accuracy. The IQ vector is extracted from the in-phase and quadrature components of discrete complex signal, which are combined to form a two-dimensional real-valued vector, i.e.,

$$\mathbf{x} = \begin{bmatrix} \mathbf{r}_I^T \\ \mathbf{r}_Q^T \end{bmatrix} = \begin{bmatrix} r_I[0] & r_I[1] & \cdots & r_I[L-1] \\ r_Q[0] & r_Q[1] & \cdots & r_Q[L-1] \end{bmatrix}, \tag{6}$$

where $\mathbf{r}_I^T \in \mathbb{R}^L$, $\mathbf{r}_Q^T \in \mathbb{R}^L$, and $\mathbf{x} \in \mathbb{R}^{2\times L}$.

B. Problem Formulation

In semi-supervised AMR, the signal dataset $\mathcal{D}$ is divided into a training set $\mathcal{D}_{tr} = \{\mathcal{D}_{ld}, \mathcal{D}_{ud}\}$ and a test set $\mathcal{D}_{te} = \{\mathbf{x}_{te}\}$. $\mathcal{D}_{ld} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_N, y_N)\}$ is the labeled dataset and $\mathcal{D}_{ud} = \{\mathbf{x}_{N+1}, \ldots, \mathbf{x}_{N+M}\}$ is the unlabeled dataset. Generally, $M \gg N$. The purpose of semi-supervised learning is to learn a predictive model $g_\phi(f_\theta(\mathbf{x})) : X \to Y$ by using $\mathcal{D}_{ld}$ and $\mathcal{D}_{ud}$ to assign the correct modulation class for signals in $\mathcal{D}_{te}$, where θ and ϕ are the encoder and classifier parameters, respectively.

The total loss function L_t of semi-supervised learning can be expressed as a weighted sum of the unsupervised loss L_u and supervised loss L_s, i.e.,

$$L_t = \alpha \sum_{\mathbf{x}\in\mathcal{D}_{ud}} L_u(\mathbf{x}, \theta, \phi) + \beta \sum_{(\mathbf{x},\mathbf{y})\in\mathcal{D}_{ld}} L_s(\mathbf{x}, \mathbf{y}, \theta, \phi), \tag{7}$$

where α and β denote the trade-off weights.

IV. The Proposed Framework

As shown in Fig. 1, the proposed TcssAMR involves two stages of updating. In the self-supervised contrastive pre-training stage, the general features of unlabeled signals are extracted by CL, and the Transformer-based encoder is pre-trained. In the supervised fine-tuning stage, a small amount of labeled signals is used to fine-tune the CTDNN consisting of the pre-trained encoder and a randomly initialized classifier. Since the encoder can extract general features in pre-training, hierarchical learning rates are exploited to update the network parameters in fine-tuning. Compared with the semi-supervised AMR methods based on pseudo-labels, TcssAMR focuses on fully utilizing unlabeled samples to achieve higher recognition performance with fewer labeled samples.

This section describes the self-supervised contrastive pre-training, the Transformer-based encoder, the supervised fine-tuning, and the overall process of TcssAMR.

A. Self-Supervised Contrastive Pre-Training

The pre-training stage mainly consists of four components, as shown in Fig. 1. The unlabeled signals is augmented through time warping to generate positive samples. Then, the original and augmented IQ signals are respectively passed through the encoder $f_\theta(\cdot)$ to obtain the feature vector. Subsequently, a projection head $g_\xi(\cdot)$ composed of nonlinear fully connected layers is used to obtain a lower-dimensional

Authorized licensed use limited to: Rowan University Libraries. Downloaded on November 03,2025 at 22:28:36 UTC from IEEE Xplore. Restrictions apply.

projection vector. Finally, the parameters of the pre-training model are updated by the contrastive loss function.

1) Time Warping: In [21], three augmentation methods of rotation, reverse and adding noise were compared with AP vector as an input, and the rotation achieved best results. IQ signal is an in-phase quadrature sequence transformed with time, and different modulation patterns make the signal show different variation trends. Considering the time correlation of IQ signal, stochastic data augmentation methods of time series are compared in this study, including time warping, convolution, time shifting, dropping and pooling [33]. Taking the QPSK signal as an example, the visualization of various augmentation methods is presented in Fig. 2. Although the rotation does not change the amplitude of signal, the waveform of the IQ signal has been completely changed. Compared with other methods, time warping changes the sampling rate of some segments while maintaining the original signal characteristics.

Time warping randomly changes the speed of timeline through a number of speed changes and the maximum ratio of maximum/minimum speed. Its main idea is to use piecewise interpolation to expand and compress the time series without changing their length [34]. Time warping changes the sampling frequency of the signal and the path delay of channel, which is closer to the changes in signal processing than other augmentation methods. The experimental results show that time warping is the most effective augmentation method (see Section V-B for details).

For the signal $\mathbf{x} \in \mathbb{R}^{2\times L}$ in the unlabeled training set $\mathcal{D}_{ud}$, the augmented signal $\widehat{\mathbf{x}}_i \in \mathbb{R}^{2\times L}$ is obtained after time warping. The signals are respectively passed through the Transformer-based encoder $f_\theta(\cdot)$ with the same parameters, and the feature vectors with a dimension of $d_{model} = 80$ are output. The structure of the Transformer-based encoder is described in detail in Section IV-B.

2) Projection Head: The ablation experiments in [20] showed that introducing a learnable nonlinear transformation between the feature representation and contrastive loss can improve the quality of learned representation significantly. Similar results were observed in [30], [31], [32], although the number of layers and the output size of projection head were different. This study chooses two fully connected layers and the ReLU function to implement feature projection, i.e.,

$$g_\xi(\mathbf{e}_i) = \left(\max\left(0, \mathbf{e}_i \mathbf{W}_1^{\text{proj}}\right)\right)\mathbf{W}_2^{\text{proj}}, \tag{8}$$

where $\mathbf{e}_i$ is the encoded vector, $\mathbf{W}_1^{\text{proj}} \in \mathbb{R}^{80\times 128}$ and $\mathbf{W}_2^{\text{proj}} \in \mathbb{R}^{128\times 64}$ are the parameter matrices.

3) Contrastive Loss: For a sample $\mathbf{x}_i \in \mathcal{D}_{ud}$, the projection vector $\mathbf{v}_i = g_\xi(f_\theta(\mathbf{x}_i))$ is output after the encoder and projection head. Similarly, the projection vector $\widehat{\mathbf{v}}_i$ of the augmented sample $\widehat{\mathbf{x}}_i$ is obtained. For a mini-batch of B_u samples, $2B_u$ samples are obtained through time warping. Each sample and its augmented sample are mutually positive, while the other $2(B_u - 1)$ samples are negative. The contrastive loss

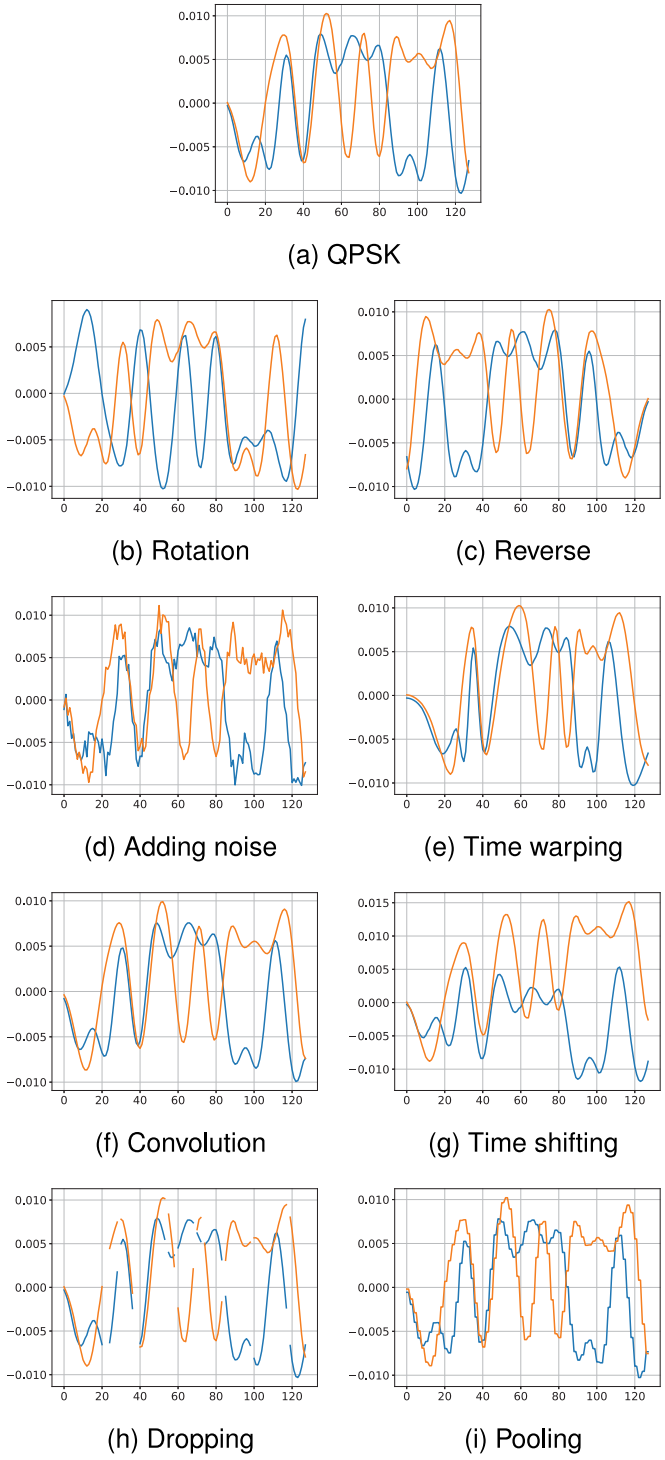


Fig. 2. Visualization of various augmentation methods for QPSK signal.

function of $\mathbf{v}_i$ and $\widehat{\mathbf{v}}_i$ is represented as:

$$\ell(\mathbf{v}_i, \widehat{\mathbf{v}}_i) = -\log \frac{\exp\left(\frac{\text{sim}(\mathbf{v}_i, \widehat{\mathbf{v}}_i)}{\tau}\right)}{\sum_{\substack{k=1,\\ k\neq i}}^{B_u} \exp\left(\frac{\text{sim}(\mathbf{v}_i, \mathbf{v}_k)}{\tau}\right) + \sum_{k=1}^{B_u} \exp\left(\frac{\text{sim}(\mathbf{v}_i, \widehat{\mathbf{v}}_k)}{\tau}\right)}, \tag{9}$$

where τ is a temperature parameter, and $\text{sim}(\mathbf{u}, \mathbf{v}) = \mathbf{u}^T\mathbf{v}/\|\mathbf{u}\|\|\mathbf{v}\|$ denotes the $l2$-normalized similarity of $\mathbf{u}$ and $\mathbf{v}$. The loss function for the batch samples updating parameters

Authorized licensed use limited to: Rowan University Libraries. Downloaded on November 03,2025 at 22:28:36 UTC from IEEE Xplore. Restrictions apply.

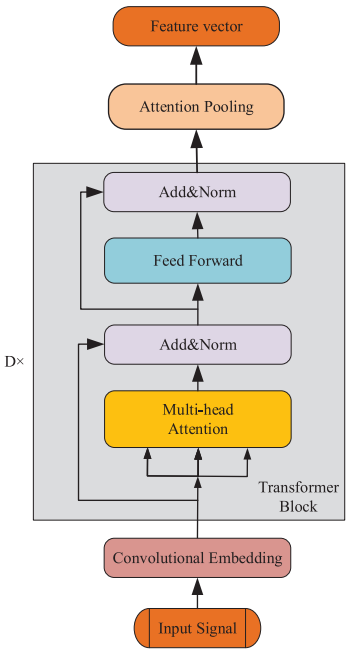


Fig. 3. The structure of the Transformer-based encoder.

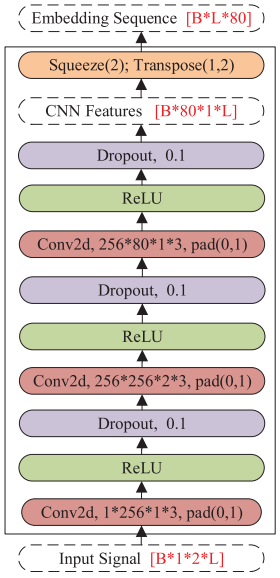


Fig. 4. The structure of convolutional embedding.

θ and ξ is:

$$L_u(\theta,\xi)=\frac{1}{2B_u}\sum_{i=1}^{B_u}[\ell(\mathbf{v}_i,\hat{\mathbf{v}}_i)+\ell(\hat{\mathbf{v}}_i,\mathbf{v}_i)]. \tag{10}$$

B. Transformer-Based Encoder

The structure of the Transformer-based encoder $f_\theta(\cdot)$ is shown in Fig. 3. The signal passes through the convolutional embedding formed by stacking multiple convolutional layers to obtain a sequence of token embeddings. Then, global feature extraction is performed by using D identical Transformer blocks. The encoded sequence is represented as an efficient low-dimensional feature vector after attention pooling.

1) Convolutional Embedding: As shown in Fig. 4, the convolutional embedding is composed of three convolutional groups, each of which contains a two-dimensional convolutional layer, a ReLU function and a dropout. The first convolutional layer extracts the local temporal correlations of the I and Q channels respectively and maps them to a high-dimensional space. The remaining two convolutional layers extract the local correlations of IQ signal and reduce the dimension of channels. Further, the ReLU function is used to improve the nonlinear fitting ability of the model, and the dropout is used to prevent overfitting of the model by randomly deactivating the network parameters.

After the convolution groups, the low-dimensional signal is mapped to a high-dimensional space to satisfy the condition of multi-head attention calculation in the Transformer. The output of convolutional embedding can be expressed as:

$$\mathbf{z}_i=\mathrm{ConvEmbed}(\mathbf{x}_i),\quad \mathbf{z}_i\in\mathbb{R}^{L\times d_{\mathrm{model}}}, \tag{11}$$

where the length of embedding sequence remains L through the zero padding in the convolution, and the dimension of embedding vector is $d_{model}=80$.

Images have the characteristics of local correlation and translation invariance, and the convolutional layer captures the local characteristics of images through the sliding calculation of multiple convolution kernels. IQ signals have similar characteristics in the time dimension, so the CNN model can be used for AMR. CNN-based AMR requires stacking multiple convolutional layers and pooling layers to capture global features of data. In contrast, Transformer uses a self-attention mechanism to directly extract global information of data. However, the global modeling ability of the Transformer requires more data to achieve better performance. The model combining CNN and Transformer can extract local and global features of signals, thereby improving the accuracy of AMR.

2) Transformer Block: Each Transformer block mainly contains two sub-layers of a multi-head attention $\mathrm{MSA}(\cdot)$ and a fully connected feed-forward network $\mathrm{MLP}(\cdot)$ to capture the temporal and spatial information of embedding sequence, respectively. Each sub-layer integrates dropout, residual connection and layer normalization to avoid vanishing gradients and make the model more stable. The Transformer block is represented as:

$$\begin{aligned}\mathbf{z}_i'&=\mathrm{LN}(\mathrm{Drop}(\mathrm{MSA}(\mathbf{z}_i))+\mathbf{z}_i),\\ \mathbf{z}_i''&=\mathrm{LN}\big(\mathrm{Drop}\big(\mathrm{MLP}\big(\mathbf{z}_i'\big)\big)+\mathbf{z}_i'\big),\end{aligned} \tag{12}$$

where $\mathbf{z}_i''\in\mathbb{R}^{L\times d_{model}}$ is the output of the block, Drop and LN denote the dropout and layer normalization, respectively.

3) Multi-Head Attention: In the convolutional layer of CNN, multiple kernels are used to obtain different types of features such as color depth and contour. Similarly, there are H heads in multi-head attention, as shown in Fig. 5. Each head calculates the self-attention of input sequence and obtains the correlation of token vector at each position with those at other positions. The output of self-attention with H heads is concatenated and linearly projected to obtain the sequence considering

Authorized licensed use limited to: Rowan University Libraries. Downloaded on November 03,2025 at 22:28:36 UTC from IEEE Xplore. Restrictions apply.

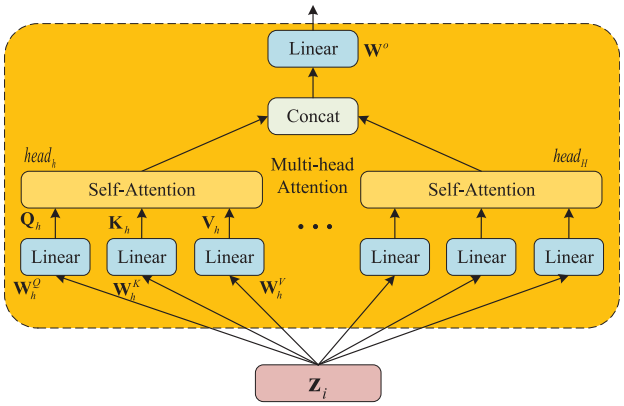


Fig. 5. The structure of multi-head attention.

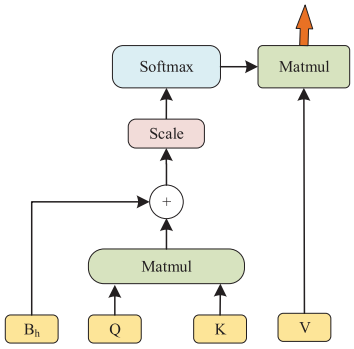


Fig. 6. The structure of self-attention mechanism with attention bias.

global correlation. This process is represented as:

$$\begin{aligned} \mathrm{MSA}(\mathbf{z}_i) &= \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_H)\mathbf{W}^O \\ with\ \mathrm{head}_h &= \mathrm{Attention}\left(\mathbf{z}_i\mathbf{W}_h^Q, \mathbf{z}_i\mathbf{W}_h^K, \mathbf{z}_i\mathbf{W}_h^V\right), \end{aligned} \quad (13)$$

where $\mathbf{W}^O \in \mathbb{R}^{Hd_h \times d_{model}}$, $\mathbf{W}_h^Q \in \mathbb{R}^{d_{model} \times d_q}$, $\mathbf{W}_h^K \in \mathbb{R}^{d_{model} \times d_k}$ and $\mathbf{W}_h^V \in \mathbb{R}^{d_{model} \times d_v}$ are the parameter matrices of linear transformation. In this study, $d_h = d_{model}/H = d_q = d_k = d_v$ is set.

As shown in Fig. 6, the self-attention mechanism calculates the correlation between input vectors in the form of a scaled dot product, and obtains representation of corresponding position by a weighted summation of value vectors. The self-attention of head_h can be represented as:

$$\mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T + \mathbf{B}^h}{\sqrt{d_k}}\right)\mathbf{V}, \quad (14)$$

where d_k is the dimension of key vector for scaling the dot products, and $\mathbf{B}^h$ is the attention bias matrix for adding positional information. The query matrix $\mathbf{Q}$, key matrix $\mathbf{K}$ and value matrix $\mathbf{V}$ are linear projections of the input sequence.

Due to the parallel processing mechanism of self-attention in the Transformer, it is necessary to add positional information at different locations of the input sequence. The common solution is to add an absolute sinusoidal [23] or learnable relative positional encoding [24] to the embedding sequence before the transformer block. However, both methods suffer from signal masking due to the uncertainty of signal amplitude, and the positional information is lost as the number of Transformer encoder layers increases.

To solve this problem, this study employs attention bias [35] to explicitly inject relative position information into each attention block. Each head has L parameters corresponding to different position offsets. The scalar attention value between two positions $x \in [1, \ldots, L]$ and $y \in [1, \ldots, L]$ is calculated as $\mathbf{Q}_{x,:}\mathbf{K}_{y,:}^T + \mathbf{BD}_{|x-y|}^h$, where $\mathbf{BD}^h$ is the positional bias dictionary of length L generated by learning.

4) Feed-Forward Network: In the feed-forward network, two linear fully connected layers further extract features in the form of dimension increase and reduction, and the ReLU function is used to ensure the nonlinearity of the network. The specific formula is:

$$\mathrm{MLP}(\mathbf{z}_i') = \max\left(0, \mathbf{z}_i'\mathbf{W}_1^{\mathrm{mlp}}\right)\mathbf{W}_2^{\mathrm{mlp}}, \quad (15)$$

where $\mathbf{W}_1^{\mathrm{mlp}} \in \mathbb{R}^{d_{model} \times d_{mlp}}$ and $\mathbf{W}_2^{\mathrm{mlp}} \in \mathbb{R}^{d_{mlp} \times d_{model}}$ are the parameter matrices of linear transformation. To reduce the complexity of model, $d_{mlp} = 2d_{model} = 160$ is set in this study.

5) Attention Pooling: After the D layers of Transformer, the encoded sequence $\mathbf{o}_i \in \mathbb{R}^{L \times 80}$ is output. In the serial LSTM, the feature vector representing global information is output after the vector at the last moment is input. For a Transformer, the output sequence has the same length as the input token sequence, and it needs to be transformed into an LSTM-like output to be further processed by the projection head and classifier.

One simple method is to flatten the encoded sequence and then output the low-dimensional feature vector through fully connected layer [6]. However, the flattening of sequence greatly increases the number of parameters and training difficulty. Some models add class token in front of the input sequence [7] or pool the output sequence [8], but these methods suffer from the loss of feature information. This study obtains the dynamic weights of different positions in the encoded sequence based on the SE module [36], and a more effective low-dimensional coding vector is output in the form of adaptive weighted summation, as shown in Fig. 7. Finally, the feature vector $\mathbf{e}_i = f_\theta(\mathbf{x}_i)$ is obtained.

C. Supervised Fine-Tuning

In the supervised fine-tuning stage, the CTDNN consisting of the pre-trained encoder and a randomly initialized classifier is updated with a small number of labeled samples $(\mathbf{x}_i, y_i) \in \mathcal{D}_{ld}$. The structure of classifier $g_\phi(\cdot)$ is the same as that of the projection head, except that the output dimension of the second layer is equal to the number of signal classes. The encoder can perform general feature extraction after pre-training, while the classifier completely relies on labeled samples to update network parameters. The encoder is fine-tuned at a smaller learning rate than the classifier. The loss function for supervised fine-tuning of the CTDNN with a batch size of B_s is represented as:

$$L_s(\theta, \phi) = -\frac{1}{B_s}\sum_{i=1}^{B_s} y_i \log(\tilde{\mathbf{y}}_i),\ \tilde{\mathbf{y}}_i = \mathrm{Softmax}\left(g_\phi(f_\theta(\mathbf{x}_i))\right). \quad (16)$$

Authorized licensed use limited to: Rowan University Libraries. Downloaded on November 03,2025 at 22:28:36 UTC from IEEE Xplore. Restrictions apply.

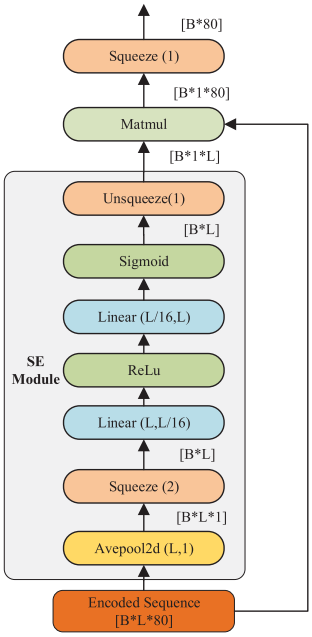


Fig. 7. The structure of attention pooling.

D. Overall Process of TcssAMR

The overall training process for TcssAMR is shown in Algorithm 1. First, the network structure, optimizer, and iterator are designed according to the hyperparameters. Then, in the pre-training stage, the encoder and projection head are updated simultaneously using unlabeled samples and contrastive loss; the encoder parameters are saved when the number of epochs is reached. In the fine-tuning stage, the encoder and classifier are updated with hierarchical learning rates through labeled samples and cross-entropy loss. After training, the parameters of CTDNN are saved to recognize IQ signals on the test set.

Algorithm 1 The Pseudocode for TcssAMR Update

Input: labeled training set $\mathcal{D}_{ld} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_N, y_N)\}$, unlabeled training set $\mathcal{D}_{ud} = \{\mathbf{x}_{N+1}, \ldots, \mathbf{x}_{N+M}\}$, and hyperparameters $D, H, B_s, B_u, lr_1, lr_2, lr_3$.

Output: model parameters θ, ϕ.

Initial the parameters of encoder $f_\theta(\cdot)$ and projection head $g_\xi(\cdot)$.

repeat

for sampled minibatch $\{\mathbf{x}_i\}_{i=1}^{B_u}$ from dataset $\mathcal{D}_{ud}$ **do**

Time warping augmentation obtains $\{\widehat{\mathbf{x}}_i\}_{i=1}^{B_u}$.

for all $i \in \{1, \ldots, B_u\}$ **do**

$\mathbf{e}_i = f_\theta(\mathbf{x}_i), \mathbf{v}_i = g_\xi(\mathbf{e}_i)$.

$\widehat{\mathbf{e}}_i = f_\theta(\widehat{\mathbf{x}}_i), \widehat{\mathbf{v}}_i = g_\xi(\widehat{\mathbf{e}}_i)$.

Calculate the contrastive loss function $L_u(\theta, \xi)$.

Update networks f_θ and g_ξ with the same learning rate lr_1.

until iterative epochs=100, save the parameters of encoder $f_\theta(\cdot)$.

Initial the parameters of classifier $g_\phi(\cdot)$.

repeat

for sampled minibatch $\{\mathbf{x}_i, y_i\}_{i=1}^{B_s}$ from dataset $\mathcal{D}_{ld}$ **do**

$\mathbf{e}_i = f_\theta(\mathbf{x}_i), \tilde{\mathbf{y}}_i = \mathrm{Softmax}(g_\phi(\mathbf{e}_i))$.

Calculate the cross entropy loss function $L_s(\theta, \phi)$.

Update networks f_θ and g_ϕ with learning rates lr_2 and lr_3 respectively.

until iterative epochs = 50, save the parameters of CTDNN (f_θ and g_ϕ).

V. Experiments

In this section, the effectiveness of TcssAMR is verified through linear evaluation and fine-tuning. Linear evaluation indicates that the parameters of the encoder are frozen after pre-training with unlabeled samples, and labeled samples are only used to update the classifier. Fine-tuning indicates that labeled samples are used to update the entire CTDNN model, and the TcssAMR obtains better recognition results with hierarchical learning rates. The RML2016.10a dataset is used for linear evaluation of self-supervised contrastive pre-training in terms of data augmentation, encoder, and CL model, thus verifying the feature representation ability of TcssAMR. To verify the performance of TcssAMR, it is compared with existing semi-supervised AMR methods on the RML2016.10a and RML2016.10b datasets.

The signals in the RML2016.10a dataset are modeled with the GNU Radio, and the effects of sample rate offset, carrier frequency offset, multi-path fading channel and additive white Gaussian noise are considered [4]. The signal received by the simulation environment is filtered by a rectangular window and sampled at a rate of 1 Msamp/s. Meanwhile, there are 11 types of signals including digital modulation (8PSK, BPSK, CPFSK, GFSK, 4PAM, 16QAM, 64QAM, QPSK) and analog modulation (AM-DSB, AM-SSB, WBFM). The SNR ranges from -20 dB to 18 dB at a step of 2 dB. Each type of signals has 1000 samples per SNR. The RML2016.10b dataset is an enlarged version of the RML2016.10a dataset, and it contains 10 types of signals except for AM-SSB. Each type of signals has 6000 samples per SNR. The length of discrete signals in the two datasets is $L = 128$, and the dimension of each IQ signal is 2×128.

In this section, the details of experiment are first introduced, and then comparison and analysis experiments are conducted from four aspects: data augmentation, encoder, CL model and semi-supervised method.

A. Implementation Details

This experiment was implemented on a Windows server with the DL platform PyTorch. The server was equipped with an NVIDIA GeForce RTX 3090 GPU and an Intel Xeon Silver 4210R CPU. The proportion of the number of samples in the unlabeled dataset, validation set, and test set was 5:2:3. For the labeled dataset, N samples were randomly selected from the unlabeled dataset. In linear evaluation, the amount of labeled samples was equal to the amount of unlabeled samples. To ensure the fairness of experiment and the reliability of results, the same number of samples were randomly selected from the signals of each type under each SNR.

Authorized licensed use limited to: Rowan University Libraries. Downloaded on November 03,2025 at 22:28:36 UTC from IEEE Xplore. Restrictions apply.

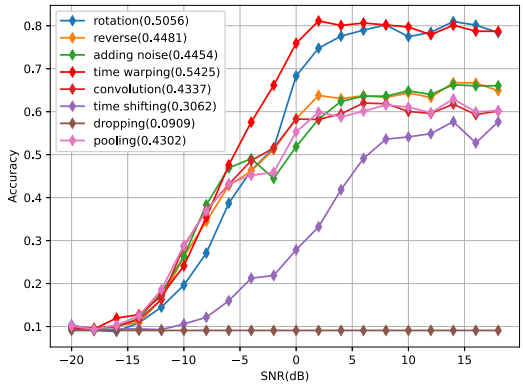


Fig. 8. The accuracy of different data augmentation methods under various SNRs.

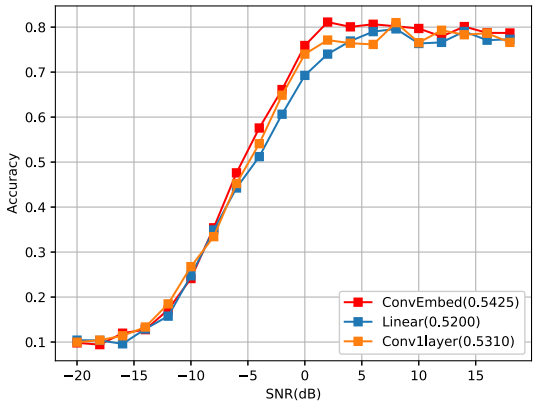


Fig. 9. The accuracy of different embedding methods under various SNRs.

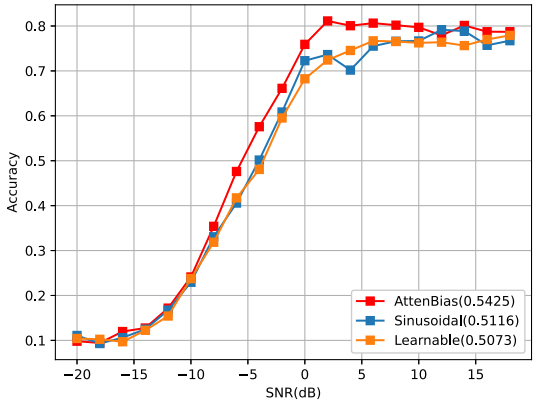


Fig. 10. The accuracy of different positional encodings under various SNRs.

In the pre-training stage, the number of training epochs was set to 100; the initial learning rate was 0.001 and it decayed by 0.1 every 50 epochs; the batch size $B_u = 1024$, and the temperature of contrastive loss was 0.07. During the stages of supervised linear evaluation or fine-tuning, training was performed for 50 epochs with a batch size of $B_s = 512$. The learning rate of the classifier in the linear evaluation was 0.0001. The learning rates of the encoder and classifier in the fine-tuning were set to 0.0005 and 0.001, respectively. The update of network model adopted the Adam optimizer, and the weight decay was 1E-6. For the network structure, the number of layers of the encoder was $D = 2$, the number of heads was $H = 8$, and the dropout was 0.1. Unless specified, the hyperparameters in the experiments adopted the aforementioned values.

B. Comparison of Data Augmentation Methods

The experiment compared the data augmentation methods shown in Fig. 2, and the accuracy under various SNRs is presented in Fig. 8. It could be seen that time warping achieved an average accuracy of 0.5425, which was 3.69% higher than the rotation. Particularly, between -10 dB and 5 dB, time warping had a more prominent effect than rotation. Rotation simulates the phase deflection caused by Doppler frequency shift. However, the angle of rotated was randomly selected from $0, \pi/2, \pi$ and $3\pi/2$, which could not effectively increase the diversity of signals. Moreover, although rotation did not change the amplitude of signal, the values of quadrature and in-phase changed significantly. The augmentation methods of reverse, adding noise, convolution and pooling achieved similar results, with an accuracy of around 0.62 under high SNRs. Time shifting and dropping resulted in lower accuracy because the internal features and information of the signal were impaired.

C. Ablation Study of Transformer-Based Encoder

To examine the feature extraction capability of the proposed Transformer-based encoder, ablation experiments were conducted involving embedding, position encoding and sequence transformation, and the results were compared with those of existing AMR models.

1) Embedding: Convolutional embedding was compared with two embedding methods: linear transformation and single-layer convolution. The linear embedding consisted of a fully connected layer with 80 hidden units and a Tanh function. The single-layer convolutional embedding consisted of a 2D convolution with the kernel (2, 5), a Tanh function, and a dropout. The results are illustrated in Fig. 9.

The accuracy of the convolution-based embedding methods exceeded 0.75 when SNR > 0 dB. By comparison, linear embedding obtained slightly lower performance especially from -4 dB to 2 dB. The randomness of sampling start time and the temporal correlation of sequence contributed to the translation invariance and local correlation of the IQ signal, so convolutional embedding achieved higher accuracy than linear embedding. In CNN, deep and narrow networks tended to extract more rich features than shallow and wide networks. Therefore, compared to single-layer convolution, three-layer convolution achieved higher accuracy with a smaller kernel size and a larger number of parameters.

2) Positional Encoding: Attention bias was compared with the commonly used absolute sinusoidal and relative learnable positional encodings. The accuracy curve under various SNRs is presented in Fig. 10. It could be seen that the average accuracy of attention bias was 3% higher than that of the other two encoding methods. Attention bias achieved the best recognition results for two reasons: the temporal relative invariance of IQ signals was modeled by adding different bias values

Authorized licensed use limited to: Rowan University Libraries. Downloaded on November 03,2025 at 22:28:36 UTC from IEEE Xplore. Restrictions apply.

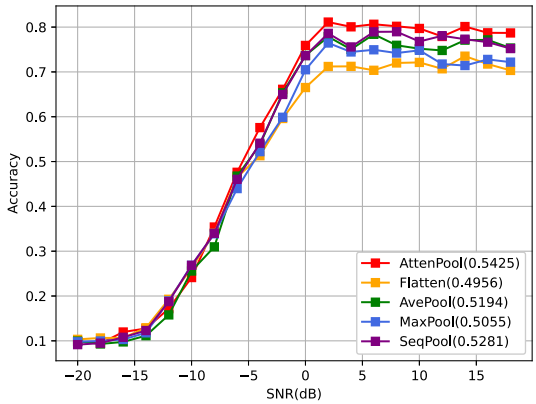


Fig. 11. The accuracy of different sequence transformation methods under various SNRs.

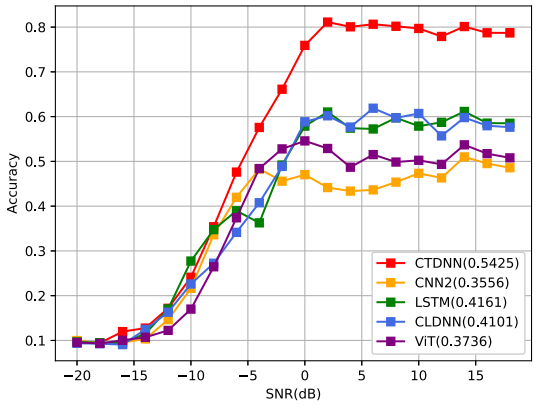


Fig. 12. The accuracy of different encoders under various SNRs.

according to relative distances; the attention bias was added in each encoder block, and the position information was retained in higher layers. Although sinusoidal positional encoding had defects such as loss of positional information and inflexibility, it did not need to be trained. Sinusoidal positional encoding achieved better results than learnable positional encoding on the relatively small dataset of modulated signals.

3) Sequence Transformation: The sequence transformation methods of flattening, average pooling, max pooling, and sequence pooling [37] were compared with attention pooling, and the dimension of feature representation was $d = 80$. Average pooling and max pooling compressed the feature values from the temporal dimension of encoded sequence. Sequence pooling and attention pooling involved weighted summation by training adaptive weights.

The accuracy of different sequence transformation methods is presented in Fig. 11. When SNR > 0 dB, the accuracy of all methods exceeded 0.7, and there was a 10% gap between the highest and lowest accuracy. Sequence pooling considered the importance of different feature vectors and obtained adaptive weights through a fully connected layer and a Softmax function. However, attention pooling further used the squeeze-and-excitation module to fully extract credible weights, so its average recognition accuracy was 1.44% higher. Average pooling obtained an average accuracy of 0.5194, while max pooling was only 0.5055. The reason was that most of feature information was lost by selecting the maximum value in the encoded sequence of length 128. Compared with the aforementioned methods, flattening converted the encoded sequence into a 10240-dimensional vector, which yielded 819,200 parameters for the linear transformation. This brought great difficulty in training the network model and extracting effective feature information, so the average accuracy was only 0.4956.

4) Comparison of Encoders: The Transformer-based encoder (CTDNN) was compared with existing AMR models of CNN2 [4], CLDNN [5], ViT [8] and LSTM [9], where all layers excepted the last fully connected layer act as an encoder. In the ViT, the patch size was set to 2, and the embedded dimension of each patch was set to 80. After two layers of Transformer for feature extraction, the output of class token was used as the feature vector. The accuracy comparison of the encoders under different SNRs is illustrated in Fig. 12. It could be seen that CNN2 achieved lowest accuracy, and ViT achieved higher accuracy than CNN2 when SNR > -4 dB. LSTM and CLDNN improved the accuracy of AMR through the implementation of RNN with time series modeling capability. The proposed Transformer-based encoder achieved 20% higher accuracy than CLDNN when SNR > 0 dB.

The ViT model improved the feature extraction capability of Transformer through pre-training on a large amount of samples, and excellent results were obtained on the ImageNet dataset [24]. However, directly applying randomly initialized ViT on semi-supervised AMR only led to an average accuracy of 0.3736. The main reason for the poor recognition results of ViT was that the IQ signal was divided into patches of equal size, and then the flattening and fully connected layer were used to convert these patches into embedded tokens, which increased the difficulty of extracting local information; meanwhile, the learnable class token was added to the first position of the embedding sequence to represent the global feature information, and the feature representation was incomplete in a small number of samples; besides, the relative learnable position encoding was only added to the embedding sequence. For the above shortcomings of ViT, this study improved the performance of Transformer in AMR from three aspects: embedding, positional encoding, and sequence transformation.

D. Comparison of CL Models

CL improved the general feature extraction ability of the encoder through stochastic data augmentation, Siamese network architecture, and contrastive loss. In this study, the performances of semi-supervised AMR with SimCLR [20], MoCoV2 [30], MoCoV3 [31] and BYOL [32] were compared. Specially, this study conducted a linear evaluation of the aforementioned CL models with the Transformer-based encoder and time-warping augmentation. The momentum update coefficient of the encoder was set to 0.99, and the length of memory bank was 65536. The accuracy curve is shown in Fig. 13. SimCLR and MoCoV3 achieved similar performance, while BYOL obtained slightly lower accuracy when SNR ≥ 0 dB. The dataset used for modulation recognition in this study was relatively small, and IQ signals required less memory than images, so the advantages of MoCoV2 were not reflected.

Authorized licensed use limited to: Rowan University Libraries. Downloaded on November 03,2025 at 22:28:36 UTC from IEEE Xplore. Restrictions apply.

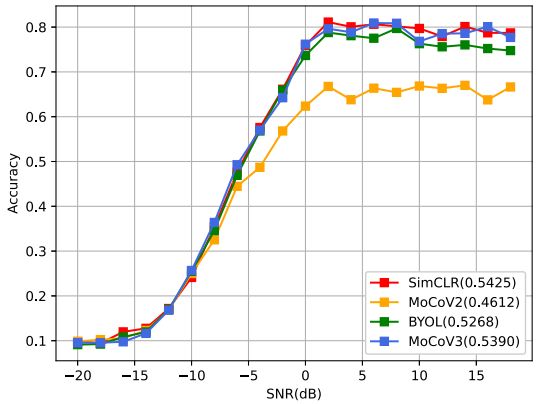


Fig. 13. The accuracy of different CL models under various SNRs.

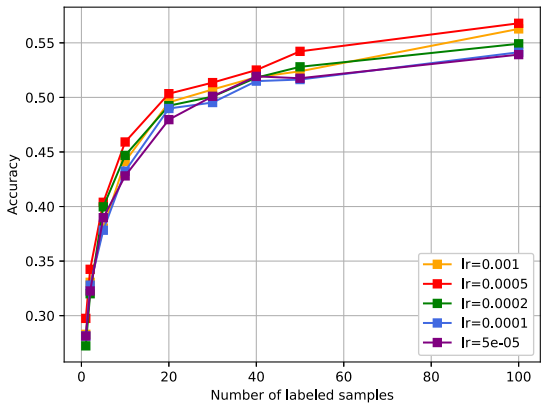


Fig. 14. The accuracy of the TcssAMR under different learning rates of encoder fine-tuning and different amount of labeled samples.

The experimental results showed that SimCLR with shared parameters and projection head was highly effective, and the prediction model without negative samples had good results.

E. Comparison of Semi-Supervised AMR Methods

Three experiments were conducted to analyze the accuracy when the amount of labeled samples $N = 1, 2, 5, 10, 20, 30, 40, 50, 100$. In the first experiment, the effect of different learning rates on the fine-tuning of encoder was analyzed. The other two experiments involved comparisons of the accuracy of different AMR methods on the RML2016.10a and RML2016.10b datasets.

1) Learning Rate For Encoder Fine-Tuning: Limited by the amount of labeled samples, the learning rate of the classifier was set to 0.001. If the same learning rate was set for the encoder, the general feature extraction ability may be destroyed. Therefore, the learning rate of the classifier was left unchanged, and the recognition accuracy of the encoder under different learning rates was analyzed.

The accuracy of the encoder at different learning rates is shown in Fig. 14. When the learning rate was 0.0005, the best recognition accuracy was achieved regardless of the amount of labeled samples for fine-tuning. When the learning rate was 0.001, the influence of labeled samples on the parameter update of the encoder had enhanced. However, a small amount of labeled samples could not make the encoder fully learn category features, and it also destroyed the feature distribution

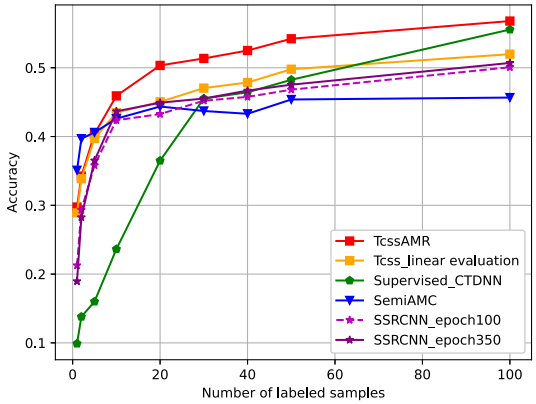


Fig. 15. The accuracy of AMR methods on the RML2016.10a dataset with different amount of labeled samples.

learned by pre-training. When the learning rate was less than 0.0005, the encoder focused on the general feature extraction and the lack of induction on same category of signals led to lower recognition accuracy.

2) Performance Comparison of Different Methods on RML2016.10a: The proposed TcssAMR was compared with SSRCNN [17], SemiAMC [22], supervised CTDNN and linear evaluation of only updating the classifier. The average accuracy of the different methods is shown in Fig. 15. When $N \leq 50$, compared to the CTDNN with supervised training, better performance could be achieved regardless of whether the entire model was to be fine-tuned or only the classifier was to be updated. The average accuracy was more than 10% higher especially when $N < 30$. It was verified that TcssAMR effectively improved the recognition accuracy by utilizing unlabeled samples in the case of a small amount of labeled samples. The performances of linear evaluation and fine-tuning were close when $N = 1, 2, 5$. As the amount of labeled samples increased, fine-tuning helped the encoder to improve the effective representation of category features, and it achieved consistently higher accuracy (about 5%).

SemiAMC obtained higher accuracy than TcssAMR when $N = 1, 2$. As the amount of labeled samples increased, the accuracy of SemiAMC increased slowly, and the accuracy stabilized around 0.45 finally. Note that SemiAMC and TcssAMR both adopted the contrastive pre-training of SimCLR. TcssAMR greatly improved the accuracy of AMR through time warping, improved Transformer encoder and fine-tuning with hierarchical learning rates. SSRCNN performed semi-supervised AMR by generating pseudo-labels on unlabeled samples. With the continuous increased in the amount of labeled samples, the confidence of the pseudo-labels continued to increase, and the accuracy on the test set was also improved. However, the accuracy was still lower than that of TcssAMR, even after training for 350 epochs.

When $N = 5, 10, 50, 100$, the confusion matrices of TcssAMR on the test set with SNR = 18dB are shown in Fig. 16. It could be seen that when $N = 5$, the probability of misjudgment between signals was high, and better recognition accuracy was achieved for BPSK, GFSK, CPFSK, and 4PAM signals. As the amount of labeled samples increased, the accuracy of some signals reached almost 100%. When

Authorized licensed use limited to: Rowan University Libraries. Downloaded on November 03,2025 at 22:28:36 UTC from IEEE Xplore. Restrictions apply.

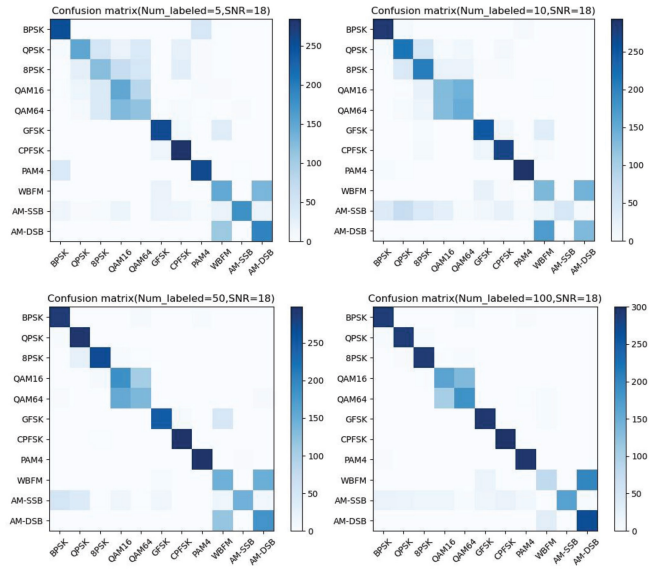


Fig. 16. The confusion matrices of TcssAMR on the RML2016.10a dataset with SNR = 18 dB.

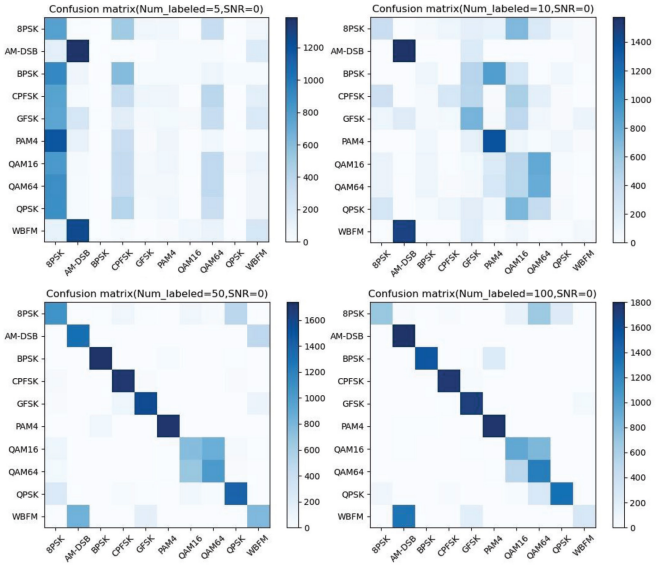


Fig. 18. The confusion matrices of TcssAMR on the RML2016.10b dataset with SNR = 0 dB.

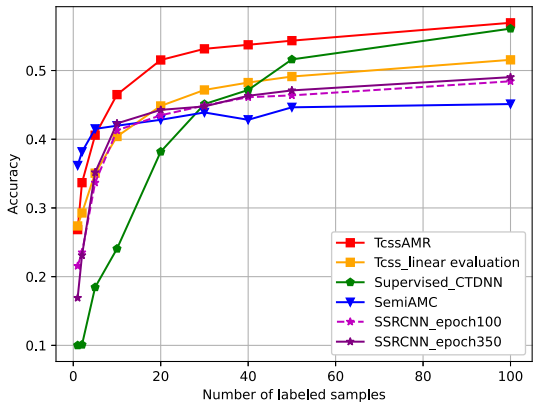


Fig. 17. The accuracy of AMR methods on the RML2016.10b dataset with different amount of labeled samples.

$N = 100$, misjudgments were still likely to occur between 16QAM and 64QAM, WBFM and AM-DSB signals, and the recognition accuracy of AM-SSB signals was relatively low. All of them could be explained from procession of generating the dataset. The observation window was small, and frequent silence between words led to misjudgment of WBFM and AM-DSB analog signals. The similarity in the modulation process between 16QAM and 64QAM signals made it difficult to distinguish them.

3) Performance Comparison of Different Methods on RML2016.10b: The recognition results on RML2016.10b are similar to those on RML2016.10a, as shown in Fig. 17. Compared to other semi-supervised and supervised learning methods, TcssAMR achieved better accuracy in all cases. The effectiveness of the proposed framework was verified again. The confusion matrices of TcssAMR on the test data with SNR = 0 dB are presented in Fig. 18. The interference of noise made it difficult for TcssAMR to extract differential features, especially when there was only a small amount of labeled samples. When $N = 5$, almost no signal could be correctly identified. When $N = 100$, the recognition accuracy was greatly improved, but there was a situation similar to that in the case of RML2016.10a. It was confirmed that more training samples were required to achieve higher accuracy at low SNR.

VI. Conclusion

In this study, TcssAMR, a semi-supervised AMR framework based on CL and Transformer, was proposed. The two-stage update approach that includes self-supervised contrastive pre-training and supervised fine-tuning effectively utilizes unlabeled signals and improves the recognition accuracy. In the fine-tuning stage, the pre-trained encoder using a smaller learning rate than the initialized classifier can further improve accuracy. Considering the temporal correlation of IQ signals, time warping is applied to TcssAMR, which has better performance than other data augmentation methods of time series. Meanwhile, the proposed CTDNN further enhances the performance of Transformer-based AMR methods. Compared with the SSRCNN and SemiAMC of the existing semi-supervised AMR methods, the average accuracy of TcssAMR is improved by more than 5% when the training set has 50 labeled samples and 500 unlabeled samples. In this work, unlabeled signals are only used for the contrastive pre-training, we believe that generating pseudo-labels for unlabeled signals after model training will further improve the accuracy of AMR.

References

[1] B. Jdid, K. Hassan, I. Dayoub, and W. H. Lim, "Machine learning based automatic modulation recognition for wireless communications: A comprehensive survey," *IEEE Access*, vol. 9, pp. 57851–57873, 2021.

[2] X. Li, F. Dong, S. Zhang, and W. Guo, "A survey on deep learning techniques in wireless signal recognition," *Wireless Commun. Mobile Comput.*, vol. 2019, 2019, Art. no. 5629572.

[3] R. Zhou, F. Liu, and C. W. Gravelle, "Deep learning for modulation recognition: A survey with a demonstration," *IEEE Access*, vol. 8, pp. 67366–67376, 2020.

[4] T. J. O'Shea, J. Corgan, and T. C. Clancy, "Convolutional radio modulation recognition networks," in *Proc. Int. Conf. Eng. Appl. Neural Netw.*, 2016, pp. 1–15.

Authorized licensed use limited to: Rowan University Libraries. Downloaded on November 03,2025 at 22:28:36 UTC from IEEE Xplore. Restrictions apply.

[5] S. Ramjee et al., "Fast deep learning for automatic modulation classification," 2019, *arXiv:1901.05850*.
[6] W. Kong, Q. Yang, X. Jiao, Y. Niu, and G. Ji, "A transformer-based CTDNN structure for automatic modulation recognition," in *Proc. 7th Int. Conf. Comput. Commun.*, Chengdu, China, 2021, pp. 159–163.
[7] S. Hamidi-Rad and S. Jain, "MCformer: A transformer based deep neural network for automatic modulation classification," in *Proc. IEEE Global Commun. Conf.*, Madrid, Spain, 2021, pp. 1–6.
[8] J. Cai, F. Gan, X. Cao, and W. Liu, "Signal modulation classification based on the transformer network," *IEEE Trans. Cogn. Commun. Netw.*, vol. 8, no. 3, pp. 1348–1357, Sep. 2022.
[9] S. Rajendran, W. Meert, D. Giustiniano, V. Lenders, and S. Pollin, "Deep learning models for wireless signal classification with distributed low-cost spectrum sensors," *IEEE Trans. Cogn. Commun. Netw.*, vol. 4, no. 3, pp. 433–445, Sep. 2018.
[10] S. Huang et al., "Automatic modulation classification using gated recurrent residual network," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7795–7807, Aug. 2020.
[11] M. Kulin, T. Kazaz, I. Moerman, and E. De Poorter, "End-to-end learning from spectrum data: A deep learning approach for wireless signal identification in spectrum monitoring applications," *IEEE Access*, vol. 6, pp. 18484–18501, 2018.
[12] Z. Liang, M. Tao, L. Wang, J. Su, and X. Yang, "Automatic modulation recognition based on adaptive attention mechanism and ResNeXt WSL model," *IEEE Commun. Lett.*, vol. 25, no. 9, pp. 2953–2957, Sep. 2021.
[13] S. Lin, Y. Zeng, and Y. Gong, "Learning of time-frequency attention mechanism for automatic modulation recognition," *IEEE Wireless Commun. Lett.*, vol. 11, no. 4, pp. 707–711, Apr. 2022.
[14] X. Yang, Z. Song, I. King, and Z. Xu, "A survey on deep semi-supervised learning," 2021, *arXiv:2103.00550*.
[15] M. Li, G. Liu, S. Li, and Y. Wu, "Radio classify generative adversarial networks: A semi-supervised method for modulation recognition," in *Proc. IEEE 18th Int. Conf. Commun. Technol. (ICCT)*, Chongqing, China, 2018, pp. 669–672.
[16] Y. Zhang and Z. Zhao, "Limited data spectrum sensing based on semi-supervised deep neural network," *IEEE Access*, vol. 9, pp. 166423–166435, 2021.
[17] Y. Dong, X. Jiang, L. Cheng, and Q. Shi, "SSRCNN: A semi-supervised learning framework for signal recognition," *IEEE Trans. Cogn. Commun. Netw.*, vol. 7, no. 3, pp. 780–789, Sep. 2021.
[18] L. Ericsson, H. Gouk, C. C. Loy, and T. M. Hospedales, "Self-supervised representation learning: Introduction, advances and challenges," 2021, *arXiv:2110.09327*.
[19] A. Jaiswal, A. R. Babu, M. Z. Zadeh, D. Banerjee, and F. Makedon, "A survey on contrastive self-supervised learning," 2022, *arXiv:2011.00362*.
[20] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. Int. Conf. Learn. Represent.*, 2020, pp. 1597–1607.
[21] L. Huang et al., "Data augmentation for deep learning-based radio modulation classification," *IEEE Access*, vol. 8, pp. 1498–1506, 2020.
[22] D. Liu, P. Wang, T. Wang, and T. Abdelzaher, "Self-contrastive learning based semi-supervised radio modulation classification," in *Proc. IEEE Mil. Commun. Conf. (MILCOM)*, San Diego, CA, USA, Nov. 2021, pp. 777–782.
[23] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, Long Beach, CA, USA, 2017, pp. 5998–6008.
[24] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.
[25] S. Chang, S. Huang, R. Zhang, Z. Feng, and L. Liu, "Multitask-Learning-based deep neural network for automatic modulation classification," *IEEE Internet Things J.*, vol. 9, no. 3, pp. 2192–2206, Feb. 2022.
[26] I. J. Goodfellow et al., "Generative adversarial nets," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 27. Montreal, QC, Canada, 2014, pp. 2672–2680.
[27] P. Baldi, "Autoencoders, unsupervised learning, and deep architectures," in *Proc. ICML Workshop Unsupervised Transfer Learn.*, 2012, pp. 37–49.
[28] Z. Wu, Y. Xiong, S. X. Yu, and D. Lin, "Unsupervised feature learning via non-parametric instance discrimination," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Salt Lake City, UT, USA, 2018, pp. 3733–3742.
[29] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Seattle, WA, USA, 2020, pp. 9726–9735.
[30] X. Chen, H. Fan, R. Girshick, and K. He, "Improved baselines with momentum contrastive learning," 2020, *arXiv:2003.04297*.
[31] X. Chen, S. Xie, and K. He, "An empirical study of training self-supervised vision transformers," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 1–10.
[32] J.-B. Grill et al.,"Bootstrap your own latent: A new approach to self-supervised learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 21271–21284.
[33] B. K. Iwana and S. Uchida, "An empirical survey of data augmentation for time series classification with neural networks," 2020, *arXiv:2007.15951v4*.
[34] T. T. Um et al., "Data augmentation of wearable sensor data for Parkinson's disease monitoring using convolutional neural networks," in *Proc. ACM*, Glasgow, U.K., 2017, pp. 216–220.
[35] B. Graham et al., "LeViT: A vision transformer in ConvNet's clothing for faster inference," in *Proc. IEEE/CVF Int. Conf. Comput.Vis.*, Montreal, QC, Canada, 2021, pp. 12239–12249.
[36] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Salt Lake City, UT, USA, 2018, pp. 7132–7141.
[37] A. Hassani et al., "Escaping the big data paradigm with compact transformers," 2021, *arXiv:2104.05704*.



Weisi Kong received the M.S. degree in electronics and communication engineering from Northwestern Polytechnical University, China, in 2019. He is currently pursuing the Ph.D. degree with the School of Telecommunication Engineering, Xidian University, China. His research interests include cognitive radio, signal recognition, deep learning, and reinforcement learning.



Xun Jiao was born in 1978. He received the Ph.D. degree. He was a Senior Engineer. His research interests include intelligent cognition and complex electromagnetic environment technologies.



Yuhua Xu was born in 1993. She is currently pursuing the M.S. degree with the Academy of Military Sciences, China. Her research interests include mission planning, intelligent decision-making, deep learning, and reinforcement learning.



Bolin Zhang was born in Sichuan in 1997. He is currently pursuing the M.S. degree with the Department of National Key Laboratory of Science and Technology on Communications, University of Electronic Science and Technology of China. His research interests include machine learning and array signal processing.



Qinghai Yang (Member, IEEE) received the B.S. degree in communication engineering from the Shandong University of Technology, China, in 1998, the M.S. degree in information and communication systems from Xidian University, China, in 2001, and the Ph.D. degree in communication engineering from Inha University, South Korea, in 2007. From 2007 to 2008, he was a Research Fellow with UWB-ITRC, South Korea. Since 2008, he has been with Xidian University. His current research interests include autonomic communication, content delivery networks, LTE-A techniques, and mobile edge computing. He received the University-President Award during his Ph.D. degree.

Authorized licensed use limited to: Rowan University Libraries. Downloaded on November 03,2025 at 22:28:36 UTC from IEEE Xplore. Restrictions apply.